Bachelor of Computer Applications (Hons. AI and ML)
Semester- III

Subject: Database Management System

Assignment 9: Advanced SQL Queries Using Joins and Subqueries

Case Study: Online Food Delivery System

By
Name: AKASH P
Reg. No.: 2511021240008

Faculty Name: Renita Joy
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
October 2026

1. Introduction

Advanced reports combine related tables with JOIN operations and use subqueries where a derived set of records is needed. Each report below is aligned to the Online Food Delivery System schema (customers, restaurants, menu items, orders, order items, payments, delivery agents and reviews) used in the earlier assignments.

7. Order report with customer, restaurant, agent, and payment (JOIN)

Combines five related tables into one readable order-tracking report.

```
SELECT o.order_id, c.name AS customer, r.restaurant_name,
       a.agent_name AS delivery_agent, p.method, p.amount,
       o.order_status
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
JOIN DeliveryAgents a ON a.agent_id = o.agent_id
JOIN Payments p ON p.order_id = o.order_id
ORDER BY o.order_id;
```


```
SELECT o.order_id, c.name AS customer, r.restaurant_name,
       a.agent_name AS delivery_agent, p.method, p.amount,
       o.order_status
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
JOIN DeliveryAgents a ON a.agent_id = o.agent_id
JOIN Payments p ON p.order_id = o.order_id
ORDER BY o.order_id;
```


RESULT GRID (illustrative output from the sample dataset)

order_id	customer	restaurant_name	delivery_agent	method	amount	order_status
1	Rahul Verma	Spice Garden	Manoj Kumar	UPI	640.0	Delivered
2	Sneha Reddy	Dosa Corner	Salman Ali	Card	160.0	Delivered
3	Pooja Nair	Pizza Hub	Ravi Teja	UPI	630.0	Delivered
4	Rahul Verma	Pizza Hub	Manoj Kumar	Cash	400.0	Delivered
5	Divya Menon	Wok Express	Salman Ali	Card	420.0	Delivered
6	Sneha Reddy	Spice Garden	Ravi Teja	UPI	260.0	Cancelled
7	Pooja Nair	Dosa Corner	Manoj Kumar	UPI	230.0	Delivered
8	Divya Menon	Spice Garden	Salman Ali	Card	320.0	Out for Delivery

8. Customers with two or more orders (subquery)

Uses a subquery to find customers who have ordered at least twice.

```
SELECT c.customer_id, c.name
FROM Customers c
WHERE c.customer_id IN (
  SELECT customer_id
  FROM Orders
  GROUP BY customer_id
  HAVING COUNT(*) >= 2
)
ORDER BY c.customer_id;
```


```
SELECT c.customer_id, c.name
FROM Customers c
WHERE c.customer_id IN (
  SELECT customer_id
  FROM Orders
  GROUP BY customer_id
  HAVING COUNT(*) >= 2
)
ORDER BY c.customer_id;
```


RESULT GRID (illustrative output from the sample dataset)

customer_id	name
1	Rahul Verma
2	Sneha Reddy
4	Pooja Nair
6	Divya Menon

9. Payment summary by payment status

Summarizes number and value of payments grouped by their status.

```
SELECT payment_status, COUNT(*) AS payment_count,
       SUM(amount) AS total_amount, ROUND(AVG(amount),2) AS average_amount
FROM Payments
GROUP BY payment_status
ORDER BY total_amount DESC;
```


```
SELECT payment_status, COUNT(*) AS payment_count,
       SUM(amount) AS total_amount, ROUND(AVG(amount),2) AS average_amount
FROM Payments
GROUP BY payment_status
ORDER BY total_amount DESC;
```


RESULT GRID (illustrative output from the sample dataset)

payment_status	payment_count	total_amount	average_amount
Paid	6	2480.0	413.33
Pending	1	320.0	320.0
Refunded	1	260.0	260.0

10. Undelivered orders placed before 2026-10-23

Finds orders that are still open (not delivered and not cancelled) using the assignment's sample date.

```
SELECT o.order_id, c.name, r.restaurant_name, o.order_date, o.order_status
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
WHERE o.delivery_date IS NULL
  AND o.order_status <> 'Cancelled'
  AND o.order_date < '2026-10-23'
ORDER BY o.order_date;
```

```
SELECT o.order_id, c.name, r.restaurant_name, o.order_date, o.order_status
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
WHERE o.delivery_date IS NULL
  AND o.order_status <> 'Cancelled'
  AND o.order_date < '2026-10-23'
ORDER BY o.order_date;
```

RESULT GRID (illustrative output from the sample dataset)

order_id	name	restaurant_name	order_date	order_status
8	Divya Menon	Spice Garden	2026-10-22	Out for Delivery

11. Reviews with customer and restaurant

Displays each review together with the customer and the restaurant that was rated.

```
SELECT rv.review_id, c.name, r.restaurant_name, rv.rating, rv.review_date
FROM Reviews rv
JOIN Orders o ON o.order_id = rv.order_id
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
ORDER BY rv.review_id;
```

```
SELECT rv.review_id, c.name, r.restaurant_name, rv.rating, rv.review_date
FROM Reviews rv
JOIN Orders o ON o.order_id = rv.order_id
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
ORDER BY rv.review_id;
```

RESULT GRID (illustrative output from the sample dataset)

review_id	name	restaurant_name	rating	review_date
1	Rahul Verma	Spice Garden	5	2026-10-02
2	Sneha Reddy	Dosa Corner	4	2026-10-04
3	Pooja Nair	Pizza Hub	5	2026-10-06
4	Rahul Verma	Pizza Hub	3	2026-10-10
5	Divya Menon	Wok Express	4	2026-10-13
6	Pooja Nair	Dosa Corner	4	2026-10-19

12. Order-item listing

Demonstrates the many-to-many relationship between orders and menu items resolved by the OrderItems junction table.

```
SELECT oi.order_id, m.item_name, oi.quantity
FROM OrderItems oi
JOIN MenuItems m ON m.item_id = oi.item_id
ORDER BY oi.order_id, m.item_name;
```


```
SELECT oi.order_id, m.item_name, oi.quantity
FROM OrderItems oi
JOIN MenuItems m ON m.item_id = oi.item_id
ORDER BY oi.order_id, m.item_name;
```


RESULT GRID (illustrative output from the sample dataset)

order_id	item_name	quantity
1	Butter Chicken	2
2	Idli Vada	1
2	Masala Dosa	1
3	Farmhouse Pizza	1
3	Margherita Pizza	1
4	Garlic Bread	1
4	Margherita Pizza	1
5	Chilli Chicken	1
5	Veg Hakka Noodles	1
6	Paneer Tikka	1
7	Idli Vada	2
7	Masala Dosa	1
8	Butter Chicken	1

2. Additional report: count orders per customer

```
SELECT c.customer_id, c.name, COUNT(o.order_id) AS total_orders
FROM Customers c
LEFT JOIN Orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_orders DESC, c.name;
```

This report includes customers with zero orders because it uses LEFT JOIN. It can help the platform understand ordering activity by customer.


```
SELECT c.customer_id, c.name, COUNT(o.order_id) AS total_orders
FROM Customers c
LEFT JOIN Orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_orders DESC, c.name;
```


RESULT GRID (illustrative output from the sample dataset)

customer_id	name	total_orders
6	Divya Menon	2
4	Pooja Nair	2
1	Rahul Verma	2
2	Sneha Reddy	2
5	Imran Khan	0
3	Karthik Rao	0

3. Conclusion

The reports demonstrate multi-table joins, grouping, aggregate functions, and a subquery. They can support practical tasks such as tracking open orders, reviewing customer ratings, checking payment totals, and understanding ordering activity.